

CS462 – Big Data Systems

Lab 1: Big Data File Formats – JSON, Apache Arrow & Parquet

Bellevue College – Spring 2026

Course	CS462 – Big Data Systems
Lab Number	Lab 1
Topics	JSON, Apache Arrow (in-memory & IPC File Format), Parquet, Format Comparison
Estimated Time	2–3 hours
Points	190
Prerequisite	Lab 0 – Docker Setup & Container Environment (image must be built)
Submission	Canvas – upload a single ZIP file (see Deliverables section)

Learning Objectives

By the end of this lab you will be able to:

- Start the Lab 1 Docker container and confirm the data environment is ready
- Create, write, and read JSON files using Python's built-in json module
- Build and inspect Apache Arrow in-memory tables using PyArrow
- Add computed columns and filter rows using PyArrow's compute module
- Persist an Arrow table to disk using the Arrow IPC File Format (.arrow) and read it back
- Write and read Apache Parquet files and use column pruning to read only selected fields
- Compare on-disk storage size across all three file formats and explain the trade-offs
- Generate large-scale synthetic datasets and observe how file sizes change at 10, 100 000, and 1 000 000 rows
- Compare Parquet compression codecs (NONE, Snappy, ZSTD) and explain their trade-offs
- Benchmark read performance across formats and measure the benefit of column pruning at scale

Prerequisites

- Lab 0 completed: Docker Desktop installed, lab0 image built
- The lab0 folder from Lab 0 (with Dockerfile, docker-compose.yml, and data/ subfolder)
- At least 4 GB of free RAM available for Docker
- Basic Python familiarity (variables, loops, list comprehensions)

Note: If you no longer have your Lab 0 files, refer to the Environment Setup section in the Lab 0 handout to recreate the lab0 folder and rebuild the image before continuing.

Environment Setup

Navigate to your lab0 folder from Lab 0 and start an interactive container shell:

```
docker compose run --rm lab0 bash
```

Your prompt should change to root@<id>:/lab#. Quickly verify the packages are available:

```
python3 -c "import pyarrow, pandas; print('Ready')"
```

Note: If the container fails to start, make sure Docker Desktop is running and that you are in the correct lab0 directory. If the image no longer exists, run docker compose build to rebuild it.

Part 1: Working with JSON Files (30 points)

JSON (JavaScript Object Notation) is a human-readable, text-based format widely used for data exchange between systems. In this part you will create a synthetic sales dataset, save it as a JSON file, read it back, and perform simple queries — all using Python's built-in json module.

1.1 Write the JSON File

Create the following script at `/lab/data/json_lab.py`:

`json_lab.py`:

```
import json
import os

# 1. Define a synthetic sales dataset (10 records)
records = [
    {"id": 1, "product": "Widget A", "quantity": 120, "unit_price": 9.99, "region": "West"},
    {"id": 2, "product": "Widget B", "quantity": 85, "unit_price": 14.49, "region": "East"},
    {"id": 3, "product": "Gadget X", "quantity": 200, "unit_price": 5.75, "region": "West"},
    {"id": 4, "product": "Gadget Y", "quantity": 60, "unit_price": 24.99, "region": "North"},
    {"id": 5, "product": "Widget A", "quantity": 95, "unit_price": 9.99, "region": "East"},
    {"id": 6, "product": "Gadget X", "quantity": 150, "unit_price": 5.75, "region": "South"},
    {"id": 7, "product": "Widget B", "quantity": 70, "unit_price": 14.49, "region": "West"},
    {"id": 8, "product": "Gadget Y", "quantity": 110, "unit_price": 24.99, "region": "East"},
    {"id": 9, "product": "Widget A", "quantity": 45, "unit_price": 9.99, "region": "North"},
    {"id": 10, "product": "Gadget X", "quantity": 180, "unit_price": 5.75, "region": "South"},
]

# 2. Write to a JSON file
os.makedirs("/lab/data", exist_ok=True)
with open("/lab/data/sales.json", "w") as f:
    json.dump(records, f, indent=2)
print(f"Written {len(records)} records to sales.json")
print(f"File size: {os.path.getsize('/lab/data/sales.json')} bytes")
```

Run the script:

```
python3 /lab/data/json_lab.py
```

Run a `cat` command to verify the content of the output JSON file. Then take a screenshot of the output confirming the file was written. Label it `screenshot-1a.png`.

1.2 Read, Filter, and Query the JSON File

Append the following block to `json_lab.py` and re-run:

```
# 3. Read back and inspect
with open("/lab/data/sales.json", "r") as f:
    loaded = json.load(f)
print(f"\nLoaded {len(loaded)} records")
print("First record:", loaded[0])

# 4. Filter by region
west = [r for r in loaded if r["region"] == "West"]
print(f"\nWest-region records ({len(west)} total):")
for r in west:
    print(f"    id={r['id']}    product={r['product']}    qty={r['quantity']}")

# 5. Compute total revenue
total_revenue = sum(r["quantity"] * r["unit_price"] for r in loaded)
print(f"\nTotal revenue across all records: ${total_revenue:,.2f}")
```

Note the filter predicate `["region"] == "West"` over the region attribute in step # 4.

Take a screenshot showing the filter results and total revenue. Label it screenshot-1b.png.

Part 1 – Deliverables

- screenshot-1a.png – `json_lab.py` output showing file written and size
- screenshot-1b.png – West-region filter results and total revenue

Part 2: Apache Arrow In-Memory Tables (35 points)

Apache Arrow defines a language-independent, columnar, in-memory format for analytical data. Unlike a Python list of dicts, an Arrow Table stores each column as a contiguous typed array — a layout that enables highly efficient vectorised operations without any format conversion. In this part you will load your sales data into an Arrow Table, inspect its schema, and use the PyArrow compute module to add a derived column and filter rows.

2.1 Build an Arrow Table and Inspect the Schema

Create `/lab/data/arrow_lab.py`:

`arrow_lab.py`

```
import pyarrow as pa
import pyarrow.compute as pc
import json

# 1. Load existing JSON data
with open("/lab/data/sales.json") as f:
    records = json.load(f)

# 2. Build an Arrow Table from the list of dicts
table = pa.Table.from_pylist(records)

# 3. Inspect schema and dimensions
print("Schema:")
print(table.schema)
print(f"\nRows: {table.num_rows} Columns: {table.num_columns}")
print(f"Column names: {table.column_names}")

# 4. Inspect the unit_price column
unit_prices = table.column("unit_price")
print(f"\nunit_price type : {unit_prices.type}")
print(f"unit_price values: {unit_prices.to_pylist()}")
```

Run the script:

```
python3 /lab/data/arrow_lab.py
```

Notice how PyArrow inferred column types automatically: `int64` for whole numbers, `double` for decimals, `string` for text. Take a screenshot of the schema output. Label it `screenshot-2a.png`.

2.2 Add a Computed Column and Filter Rows

Append the following to `arrow_lab.py` and re-run:

```
# 5. Compute a revenue column: quantity * unit_price
revenue = pc.multiply(table["quantity"], table["unit_price"])
```

```
table = table.append_column("revenue", revenue)
print("\nColumn names after adding revenue:", table.column_names)

# 6. Filter to West-region rows only
west_mask = pc.equal(table["region"], "West")
west_table = table.filter(west_mask)
print(f"\nWest-region rows ({west_table.num_rows} records):")
for i in range(west_table.num_rows):
    prod = west_table["product"][i].as_py()
    qty = west_table["quantity"][i].as_py()
    rev = west_table["revenue"][i].as_py()
    print(f" {prod:12s} qty={qty:4d} revenue=${rev:8.2f}")

# 7. Total revenue via Arrow compute
total = pc.sum(table["revenue"]).as_py()
print(f"\nTotal revenue (Arrow compute): ${total:,.2f}")
```

Again, note the filter predicate over the West region:

```
west_mask = pc.equal(table["region"], "West")
west_table = table.filter(west_mask)
```

Take a screenshot of the computed revenue column, the West-region filtered output, and the total. Label it screenshot-2b.png.

Note: Revenue values may show many decimal places due to binary floating-point representation (e.g. 1196.9999... instead of 1197.00). This is normal and not a bug.

Part 2 – Deliverables

- screenshot-2a.png – Arrow schema and unit_price column type
- screenshot-2b.png – Revenue column added, West-region filtered rows, and total revenue

Part 3: Persisting Data – Arrow IPC & Parquet (45 points)

In this part you will write the same Arrow Table to disk in two binary formats — Parquet (.parquet) and the Arrow IPC File Format (.arrow) — and compare their characteristics. You will also explore Parquet's column-pruning capability and finish with a three-way size comparison across all three formats from this lab.

Create a single file `/lab/data/formats_lab.py` and build it up section by section. After adding each block, re-run the whole file to see the cumulative output.

3.1 Write as Parquet and Read It Back

Start `formats_lab.py` with the following:

`formats_lab.py`

```
import pyarrow as pa
import pyarrow.ipc as ipc
import pyarrow.parquet as pq
import pyarrow.compute as pc
import json, os

# 1. Rebuild the Arrow Table (same as Part 2)
with open("/lab/data/sales.json") as f:
    records = json.load(f)
table = pa.Table.from_pylist(records)
revenue = pc.multiply(table["quantity"], table["unit_price"])
table = table.append_column("revenue", revenue)

# 2. Write as Parquet
pq.write_table(table, "/lab/data/sales.parquet")
print("Wrote sales.parquet")

# 3. Read back and confirm schema
loaded_pq = pq.read_table("/lab/data/sales.parquet")
print("\nSchema from sales.parquet:")
print(loaded_pq.schema)
print(f"Rows: {loaded_pq.num_rows} Columns: {loaded_pq.num_columns}")
```

Run the file and confirm the Parquet schema matches your Part 2 Arrow schema (plus the extra column for revenue). Take a screenshot. Label it `screenshot-3a.png`.

3.2 Write as Arrow IPC File Format and Read It Back

The Arrow IPC File Format (.arrow) serialises the Arrow in-memory layout directly to disk. Because the on-disk representation closely mirrors what is already in RAM, reading it back requires almost no decoding work, making it the fastest format for round-tripping Arrow data between processes or between a process and local storage.

Append the following block to `formats_lab.py` and re-run:

```
# 4. Write as Arrow IPC File Format
with ipc.new_file("/lab/data/sales.arrow", table.schema) as writer:
    writer.write_table(table)
print("\nWrote sales.arrow")

# 5. Read the Arrow IPC file back
with ipc.open_file("/lab/data/sales.arrow") as reader:
    loaded_arrow = reader.read_all()
print("\nSchema from sales.arrow:")
print(loaded_arrow.schema)
print(f"Rows: {loaded_arrow.num_rows} Columns: {loaded_arrow.num_columns}")
```

Note that the schema printed from `sales.arrow` is identical to the one from `sales.parquet` — both formats preserve the Arrow column types. Take a screenshot. Label it `screenshot-3b.png`.

Note: Arrow also has an IPC Streaming Format (`ipc.new_stream / open_stream`) designed for sending data over network sockets or pipes where the total size is not known upfront. The File Format used here is the random-access variant, which includes a footer index so any record batch can be read without scanning the whole file.

3.3 Column Pruning with Parquet

Parquet's columnar storage lets it skip entire columns at read time, reducing I/O for queries that only need a subset of fields. The Arrow IPC File Format does not support column pruning — the reader always loads the complete file. This distinction is important for wide tables with hundreds of columns. Append the following and re-run:

```
# 6. Column pruning: read only product, region, and revenue
subset = pq.read_table(
    "/lab/data/sales.parquet",
    columns=["product", "region", "revenue"]
)
print("\nColumn-pruned Parquet read (3 of 6 columns):")
print(f"Columns returned: {subset.column_names}")
for i in range(subset.num_rows):
    print(f"    {subset['product'][i].as_py():12s}    "
          f"{subset['region'][i].as_py():6s}    "
          f"${subset['revenue'][i].as_py():8.2f}")
```

Take a screenshot confirming only three columns were returned. Label it `screenshot-3c.png`.

3.4 Three-Way File Size Comparison

Measure the on-disk size of all three files and compute the pairwise ratios. Append this final block and re-run:

```
# 7. Three-way file-size comparison
json_size = os.path.getsize("/lab/data/sales.json")
arrow_size = os.path.getsize("/lab/data/sales.arrow")
parquet_size = os.path.getsize("/lab/data/sales.parquet")

print("\nFile size comparison:")
print(f"  JSON      : {json_size:>7} bytes")
print(f"  Arrow     : {arrow_size:>7} bytes")
print(f"  Parquet   : {parquet_size:>7} bytes")

print(f"\nArrow vs JSON:      {json_size / arrow_size:.2f}x  (JSON is larger by this factor)")
print(f"Parquet vs JSON:  {json_size / parquet_size:.2f}x  (JSON is larger by this factor)")
arrow_vs_pq = arrow_size / parquet_size
rel = 'larger' if arrow_vs_pq > 1 else 'smaller'
print(f"Arrow vs Parquet: {arrow_vs_pq:.2f}x  (Arrow is {rel} than Parquet)")
```

Record all three sizes and ratios — you will discuss the results in your reflection. Take a screenshot of the size comparison output. Label it screenshot-3d.png.

Note: With only 10 records, Parquet may actually be larger than Arrow IPC due to fixed metadata and footer overhead. This relationship reverses at large row counts where Parquet's dictionary encoding and compression dominate. Note your actual result in your write-up and explain what you observe.

Part 3 – Deliverables

- screenshot-3a.png – Parquet schema printed after reading sales.parquet
- screenshot-3b.png – Arrow IPC schema printed after reading sales.arrow
- screenshot-3c.png – Column-pruned Parquet read showing only product, region, revenue
- screenshot-3d.png – Three-way file size comparison (JSON, Arrow, Parquet) with ratios

Part 4: Scaling Up – Compression, Performance & Large Datasets (60 points)

In Parts 1–3 you worked with just 10 records. At that scale every format feels instant and Parquet's metadata overhead can make it appear larger than Arrow IPC. In this part you will scale the dataset to 1 000 000 rows and observe how file sizes, compression ratios, and read performance change dramatically. You will create a new script called `scale_lab.py` for all of Part 4.

4.1 Generate a Large Synthetic Dataset

The function below reuses the same product and region vocabulary from Parts 1–3 but randomises quantities and unit prices so that the data is realistic enough to avoid

trivially perfect compression. Create `scale_lab.py` at `/lab/data/scale_lab.py` with the following content:

`scale_lab.py`:

```
import pyarrow as pa
import pyarrow.compute as pc
import pyarrow.parquet as pq
import pyarrow.ipc as ipc
import json, os, time, random

random.seed(42)

PRODUCTS = ["Widget A", "Widget B", "Gadget X", "Gadget Y"]
REGIONS = ["West", "East", "North", "South"]
ROW_COUNTS = [10, 100_000, 1_000_000]

def generate_table(n):
    """Return an Arrow Table with n rows of randomised sales data."""
    ids = list(range(1, n + 1))
    products = [random.choice(PRODUCTS) for _ in range(n)]
    quantities = [random.randint(1, 100) for _ in range(n)]
    prices = [round(random.uniform(5.0, 500.0), 2) for _ in range(n)]
    regions = [random.choice(REGIONS) for _ in range(n)]

    table = pa.table({
        "id": ids,
        "product": products,
        "quantity": quantities,
        "unit_price": prices,
        "region": regions,
    })
    revenue = pc.multiply(table["quantity"], table["unit_price"])
    table = table.append_column("revenue", revenue)
    return table
```

The `random.seed(42)` call makes results reproducible. Every student will get the same data, which simplifies grading.

4.2 Three-Way Size Comparison at Scale

Add the following code to `scale_lab.py`. It writes each table size to JSON, Arrow IPC, and Parquet, then prints a comparison table:

```
# — 4.2 Size comparison at three scales —————
def write_all_formats(table, n):
    """Write table to JSON, Arrow IPC, and Parquet; return dict of sizes."""
    tag = f"_{n}"
    sizes = {}

    # JSON
    json_path = f"/lab/data/sales{tag}.json"
    rows = table.to_pylist()
    with open(json_path, "w") as f:
        json.dump(rows, f)
    sizes["JSON"] = os.path.getsize(json_path)
```

```

# Arrow IPC
arrow_path = f"/lab/data/sales{tag}.arrow"
with pa.OSFile(arrow_path, "wb") as f:
    writer = ipc.new_file(f, table.schema)
    writer.write_table(table)
    writer.close()
sizes["Arrow IPC"] = os.path.getsize(arrow_path)

# Parquet (default Snappy compression)
pq_path = f"/lab/data/sales{tag}.parquet"
pq.write_table(table, pq_path)
sizes["Parquet (Snappy)"] = os.path.getsize(pq_path)

return sizes

print("=" * 65)
print(f"{'Rows':<12} {'JSON':>12} {'Arrow IPC':>12} {'Parquet':>12}")
print("-" * 65)
for n in ROW_COUNTS:
    table = generate_table(n)
    sizes = write_all_formats(table, n)
    print(f"{n:<12,} {sizes['JSON']:>12,} {sizes['Arrow IPC']:>12,} {sizes['Parquet (Snappy)']:>12,}")
print("=" * 65)

```

Run the script: `python3 scale_lab.py`. You should see a formatted table showing how file sizes diverge as row counts grow. Take a screenshot of this output. Label it screenshot-4a.png.

Notice how at 10 rows Parquet is comparable to (or even larger than) Arrow IPC, but at 1 000 000 rows it is dramatically smaller. This is because Parquet's dictionary encoding compresses the repetitive string columns (product, region) and its general-purpose codec (Snappy) further shrinks the data.

4.3 Comparing Parquet Compression Codecs

Parquet supports several compression codecs. Add this code to `scale_lab.py` to compare three common options at the 1 000 000-row scale:

```

# — 4.3 Codec comparison —————
table_1m = generate_table(1_000_000)
codecs = ["NONE", "SNAPPY", "ZSTD"]

print("\nParquet codec comparison (1,000,000 rows):")
print("-" * 45)
print(f"{'Codec':<12} {'Size (bytes)':>14} {'Ratio vs NONE':>14}")
print("-" * 45)

codec_sizes = {}
for codec in codecs:
    path = f"/lab/data/sales_1m_{codec.lower()}.parquet"
    pq.write_table(table_1m, path, compression=codec.lower())
    sz = os.path.getsize(path)
    codec_sizes[codec] = sz

none_size = codec_sizes["NONE"]

```

```
for codec in codecs:
    ratio = f"{codec_sizes[codec] / none_size:.2f}x"
    print(f"{codec:<12} {codec_sizes[codec]:>14,} {ratio:>14}")
print("-" * 45)
```

Run the script again. Take a screenshot of the codec comparison table. Label it screenshot-4b.png.

ZSTD (Zstandard) typically achieves the best compression ratio but takes slightly longer to write. Snappy is the default because it provides a good balance of speed and compression. NONE disables compression entirely, giving you a baseline to see how much work the codecs are doing.

4.4 Timing: Full Reads vs Column-Pruned Reads

With 1 000 000 rows, read-time differences between formats become measurable. Add this timing code to `scale_lab.py`:

```
# — 4.4 Read-time comparison —————
ALL_COLS = None # read everything
FEW_COLS = ["product", "region", "revenue"] # read 3 of 6 columns

def time_read(label, read_fn, iterations=5):
    """Run read_fn several times; return (median_time, last_result)."""
    times = []
    for _ in range(iterations):
        start = time.time()
        result = read_fn()
        times.append(time.time() - start)
    times.sort()
    median = times[len(times) // 2]
    return median, result

print("\nRead-time comparison (1,000,000 rows, median of 5 runs):")
print("-" * 60)
print(f"{'Operation':<35} {'Time (s)':>10}")
print("-" * 60)

# Arrow IPC - always reads full file
t, _ = time_read("Arrow IPC full",
                  lambda: ipc.open_file(pa.OSFile("/lab/data/sales_1000000.arrow",
"rb")).read_all())
print(f"{'Arrow IPC - full read':<35} {t:>10.4f}")

# Parquet - full read
t, _ = time_read("Parquet full",
                  lambda: pq.read_table("/lab/data/sales_1000000.parquet"))
print(f"{'Parquet - full read':<35} {t:>10.4f}")

# Parquet - pruned read (3 columns)
t, _ = time_read("Parquet pruned",
                  lambda: pq.read_table("/lab/data/sales_1000000.parquet",
columns=FEW_COLS))
print(f"{'Parquet - 3-column pruned read':<35} {t:>10.4f}")

# JSON - full read
```

```
t, _ = time_read("JSON full",
                lambda: json.load(open("/lab/data/sales_1000000.json")))
print(f"'JSON - full read':<35} {t:>10.4f}")
print("-" * 60)
```

Run the script again. Take a screenshot of the timing table. Label it screenshot-4c.png.

You should see that JSON is by far the slowest to read because the parser must process every character of text. Arrow IPC is the fastest for a full read because its on-disk layout matches the in-memory layout—no decoding is needed. Parquet's column-pruned read is faster than its full read because it skips the columns you do not need, reading less data from disk.

4.5 Filtered Aggregation at Scale

Finally, measure how long it takes to filter and aggregate using Arrow's vectorised compute on the 1 000 000-row dataset. Add this code to `scale_lab.py`:

```
# — 4.5 Filtered aggregation benchmark —————
def arrow_aggregate(path):
    """Read Parquet, filter to West, return total revenue."""
    t = pq.read_table(path, columns=["region", "revenue"])
    mask = pc.equal(t["region"], "West")
    west = t.filter(mask)
    return pc.sum(west["revenue"]).as_py()

def json_aggregate(path):
    """Read JSON, filter to West, return total revenue."""
    with open(path) as f:
        rows = json.load(f)
    return sum(r["revenue"] for r in rows if r["region"] == "West")

print("\nFiltered aggregation: total West-region revenue (1,000,000 rows)")
print("-" * 55)

t, total = time_read("Parquet+Arrow agg",
                    lambda: arrow_aggregate("/lab/data/sales_1000000.parquet"))
print(f"'Parquet → Arrow (pruned + filter)':<40} {t:>8.4f} s")

t, json_total = time_read("JSON+Python agg",
                        lambda: json_aggregate("/lab/data/sales_1000000.json"))
print(f"'JSON → Python loop':<40} {t:>8.4f} s")

print(f"\nWest-region total revenue: {total:,.2f}")
print("-" * 55)

# Safety check: both paths must agree on the total
assert abs(total - json_total) < 0.01, \
    f"Mismatch! Parquet total={total}, JSON total={json_total}"
print("✓ Safety check passed: Parquet and JSON totals match.")
```

Run the script one final time. Take a screenshot of the aggregation results. Label it screenshot-4d.png.

The Arrow/Parquet path combines column pruning (reads only 2 of 6 columns) with vectorised C++ compute, while the JSON path must parse the entire text file and use a Python loop. The performance gap illustrates why columnar binary formats are the foundation of modern big-data systems.

Part 4 – Deliverables

- screenshot-4a.png – Three-way size comparison table at 10, 100 000, and 1 000 000 rows
- screenshot-4b.png – Parquet codec comparison (NONE, SNAPPY, ZSTD) at 1 000 000 rows
- screenshot-4c.png – Read-time comparison for Arrow IPC, Parquet (full & pruned), and JSON
- screenshot-4d.png – Filtered aggregation timing and West-region total revenue

Tearing Down the Container

Type `exit` (or press `Ctrl-D`) to leave the container shell. Because you used `--rm`, the container is removed automatically. Your data files and all Python scripts are preserved in the `data/` folder on your host computer since they are stored in the mounted volume.

Submission Instructions

Create a ZIP file named `lab1_<YourLastName>_<YourFirstName>.zip` containing:

- A write-up document (PDF or DOCX) that includes:
- Your name, student ID, and the date
- A brief description (2–3 sentences) of what you did in each Part
- The file sizes and ratios from Part 3.4 and Part 4.2
- Answers to the reflection questions below
- All screenshots (screenshot-1a through screenshot-4d)
- Your Python scripts: `json_lab.py`, `arrow_lab.py`, `formats_lab.py`, and `scale_lab.py`

Reflection Questions

Answer these questions in your write-up (2–4 sentences each):

- JSON is text-based while both Arrow IPC and Parquet are binary. What specific characteristics of each binary format explain their relative sizes in your three-way comparison? Use your Part 4 results to explain how the size ranking actually changed as the number of rows grew from 10 to 1 000 000.
- You used `pc.multiply` and `pc.equal` from `pyarrow.compute` instead of a Python for-loop. What is the advantage of using Arrow's vectorised compute functions for

operations on large datasets, and how does the columnar layout of an Arrow Table enable that advantage?

- Arrow IPC preserves the exact in-memory Arrow layout on disk, while Parquet uses its own columnar encoding and supports column pruning at read time. Given these trade-offs, describe one scenario where Arrow IPC is the better choice and one scenario where Parquet is the better choice. What factors drive each decision?
- In Part 4.3 you compared three Parquet compression codecs (NONE, SNAPPY, ZSTD). Explain the trade-off between compression ratio and write speed. In what type of workload would you choose ZSTD over Snappy, and vice versa?

Grading Rubric

Task / Deliverable	Points	Grading Criteria
Setup – Container started & environment confirmed	10	python3 -c "import pyarrow, pandas; print('Ready')" runs without errors
Part 1 – JSON file written	10	sales.json present; screenshot shows correct file-size output
Part 1 – JSON read, filter & revenue computed	20	Screenshot shows West-region filter results and correct total revenue
Part 2 – Arrow table created & schema printed	15	Screenshot shows table.schema with correct column types
Part 2 – Revenue column computed & West rows filtered	20	revenue column present; filtered table shows only West-region rows with correct values
Part 3 – Parquet written & schema read back	10	sales.parquet present; schema matches the Part 2 Arrow schema
Part 3 – Arrow IPC file written & schema read back	10	sales.arrow present; schema matches the Parquet schema
Part 3 – Column pruning demonstrated	10	Pruned read returns only the three requested columns
Part 3 – Three-way size comparison printed & discussed	15	All three file sizes and pairwise ratios printed; write-up discusses the Arrow vs Parquet result
Part 4 – Large dataset generated & size comparison	15	scale_lab.py runs; screenshot-4a shows 3-scale size table with clear Parquet advantage at 1M rows
Part 4 – Parquet codec comparison	15	screenshot-4b shows NONE, SNAPPY, ZSTD sizes; write-up notes compression vs speed trade-off
Part 4 – Read-time benchmarks	15	screenshot-4c shows timing for Arrow IPC, Parquet full, Parquet pruned, JSON
Part 4 – Filtered aggregation benchmark	15	screenshot-4d shows Parquet+Arrow vs JSON+Python timing and total revenue value
Reflection Questions	40	4 questions × 10 pts each; answers are 2–4 sentences and demonstrate understanding
TOTAL	210	

Note: The due date for this lab is posted on Canvas. Submissions received after that date will receive reduced points in accordance with the course late-work policy.